Specialized hooks

6 min read Updated 19 May 2026 Topic 45 of 77

Bookmark Mark done

16B. Specialized Hooks

Questions: 7 **Difficulty:** Mid to Senior **Companies:** Google, Amazon, Flipkart, Microsoft, Razorpay

---

Q 16.8 When do you use useLayoutEffect over useEffect?

Difficulty: Mid **Asked at:** Google, Amazon, Flipkart

`useEffect` runs after the browser paints. `useLayoutEffect` runs after DOM updates but before the browser paints. The difference matters when you need to read layout and make synchronous DOM changes to prevent visual flickering.

JSX Copy

```jsx
// useEffect — user might see a flash of wrong position
useEffect(() => {
  const rect = tooltipRef.current.getBoundingClientRect();
  tooltipRef.current.style.left = `${rect.width / 2}px`;
}, []);
// Browser paints → user sees tooltip at wrong position → effect runs → to

// useLayoutEffect — no flash, position is correct on first paint
useLayoutEffect(() => {
  const rect = tooltipRef.current.getBoundingClientRect();
  tooltipRef.current.style.left = `${rect.width / 2}px`;
```

Use `useLayoutEffect` **for:** DOM measurements that affect visual output (tooltip positioning, scroll position restoration, element dimensions). **Use** `useEffect` **for everything else** — API calls, subscriptions, logging, analytics. `useLayoutEffect` is synchronous and blocks painting, so overusing it hurts performance.

Q 16.9 What does useImperativeHandle do?

Difficulty: Mid **Asked at:** Amazon, Microsoft

`useImperativeHandle` customizes the instance value exposed to parent components when using `forwardRef`. Instead of exposing the entire DOM element, you expose specific methods.

JSX Copy

```jsx
const FancyInput = forwardRef(function FancyInput(props, ref) {
  const inputRef = useRef(null);

  useImperativeHandle(ref, () => ({
    focus: () => inputRef.current.focus(),
    clear: () => { inputRef.current.value = ''; },
    getValue: () => inputRef.current.value,
    // Parent can't access any other DOM properties
  }), []);

  return <input ref={inputRef} {...props} />;
});

// Parent usage
function Form() {
  const inputRef = useRef(null);
  return (
    <>
      <FancyInput ref={inputRef} />
      <button onClick={() => inputRef.current.focus()}>Focus</button>
```

```
  );
}
```

This is useful for component library APIs where you want to expose specific actions without giving consumers full DOM access.

Q 16.10 What is useId and why was it added?

Difficulty: Junior **Asked at:** Flipkart, Amazon

`useId` generates a unique ID that's stable across server and client renders. This prevents hydration mismatches when you need IDs for accessibility (linking labels to inputs, aria-describedby).

JSX Copy

```jsx
function FormField({ label, ...props }) {
  const id = useId();

  return (
    <div>
      <label htmlFor={id}>{label}</label>
      <input id={id} {...props} />
    </div>
  );
}

// Each instance gets a unique, stable ID (:r1:, :r2:, etc.)
// Same ID on server and client — no hydration mismatch
```

Before `useId`, developers used `Math.random()` or incrementing counters, which produced different values on server and client, causing hydration errors.

Q 16.11 How does useTransition work? What are non-urgent updates?

Difficulty: Senior **Asked at:** Google, Amazon, Flipkart, Razorpay

`useTransition` marks a state update as non-urgent. React can interrupt the non-urgent update to handle urgent updates (like typing in an input) first, keeping the UI responsive.

JSX Copy

```jsx
function SearchPage() {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState([]);
  const [isPending, startTransition] = useTransition();

  function handleChange(e) {
    const value = e.target.value;
    setQuery(value);  // URGENT: update input immediately

    startTransition(() => {
      setResults(filterLargeDataset(value));  // NON-URGENT: can be interr
    });
  }

  return (
    <>
      <input value={query} onChange={handleChange} />
      <div style={{ opacity: isPending ? 0.7 : 1 }}>
        {results.map(r => <ResultItem key={r.id} result={r} />)}
      </div>
    </>
  );
}
```

Without `useTransition`, typing in the input would feel sluggish because `filterLargeDataset` (processing 50,000 items) blocks the main thread before the input can update. With `useTransition`, the input updates instantly (urgent), and the filtering happens in the background (non-urgent). If the user types again before filtering finishes, React abandons the stale filter and starts a new one.

`isPending` is `true` while the transition is in progress — use it to show visual feedback (reduced opacity, spinner).

Q 16.12 What is useDeferredValue?

Difficulty: Senior **Asked at:** Google, Amazon

`useDeferredValue` is the value equivalent of `useTransition`. Instead of wrapping a setState call, you wrap a value — React defers updating the components that use the deferred value.

JSX Copy

```jsx
function SearchResults({ query }) {
  // deferredQuery lags behind query during rapid updates
  const deferredQuery = useDeferredValue(query);
  const isStale = query !== deferredQuery;

  // ExpensiveList renders with the deferred (slightly stale) query
  // while the input stays responsive with the current query
  return (
    <div style={{ opacity: isStale ? 0.7 : 1 }}>
      <ExpensiveList query={deferredQuery} />
    </div>
  );
}
```

useTransition vs useDeferredValue: `useTransition` wraps the state setter (you control the update). `useDeferredValue` wraps the value (you defer the consumer). Use `useTransition` when you own the setState call. Use `useDeferredValue` when you receive a prop and want to defer rendering based on it.

Q 16.13 What is useSyncExternalStore? When do you need it?

Difficulty: Senior Asked at: Google, Microsoft

Frontend System Desi...

`useSyncExternalStore` subscribes to external stores (anything outside React's state system) safely. It handles the edge cases of concurrent rendering that custom subscription patterns (useEffect + setState) get wrong.

JSX Copy

```jsx
// Subscribing to browser online/offline status
function useOnlineStatus() {
  return useSyncExternalStore(
    // subscribe: called with a callback that should fire when the store c
    (callback) => {
      window.addEventListener('online', callback);
      window.addEventListener('offline', callback);
      return () => {
        window.removeEventListener('online', callback);
        window.removeEventListener('offline', callback);
      };
    },
    // getSnapshot: returns the current value
    () => navigator.onLine,
    // getServerSnapshot: value during SSR (no window available)
    () => true
  );
}

function StatusBar() {
  const isOnline = useOnlineStatus();
  return <span>{isOnline ? '🟢 Online' : '🔴 Offline'}</span>;
}
```

When you need it: Subscribing to browser APIs (online status, media queries, geolocation), subscribing to third-party state libraries that aren't React-aware, and building your own state management library. Zustand and Redux use `useSyncExternalStore` under the hood.

Q 16.14 What is useInsertionEffect?

`useInsertionEffect` fires before any DOM mutations. It's designed exclusively for CSS-in-JS libraries that need to inject `<style>` tags before `useLayoutEffect` reads layout.

JSX Copy

```jsx
// Used internally by CSS-in-JS libraries like styled-components, Emotion
useInsertionEffect(() => {
  const style = document.createElement('style');
  style.textContent = `.dynamic-class { color: ${color}; }`;
  document.head.appendChild(style);
  return () => document.head.removeChild(style);
}, [color]);
```

Execution order: `useInsertionEffect` → DOM mutations → `useLayoutEffect` → browser paint → `useEffect`.

You'll almost never use this directly. It exists so CSS-in-JS libraries can inject styles at the right time — before `useLayoutEffect` reads layout dimensions that depend on those styles.

QUICK CHECK — no marks, just for you.

1. Before useId existed, developers generated element IDs with Math.random() or an incrementing counter. According to the lesson, what problem did that cause in server-rendered React apps?

- ○ The IDs were not unique enough, so two inputs on the page could collide
- ○ React stripped the generated IDs from the DOM during hydration for security reasons
- ○ The IDs differed between the server render and the client render, causing hydration errors
- ○ Screen readers ignored randomly generated IDs, so htmlFor never linked the label

Check answer

spot before it snaps into place. Based on the lesson, what fixes this?

- Move the measurement into useInsertionEffect, since it fires earliest of all the effect hooks
- Keep useEffect but add the tooltip width to its dependency array so it re-runs sooner
- Move the measurement and style write into useLayoutEffect, which runs after DOM updates but before the browser paints
- Wrap the style write in startTransition so React flushes the position as an urgent update

Check answer

3. A child component receives a `query` prop from its parent and renders an expensive list from it. The child does not own the setState call that changes `query`. Per the lesson, which hook fits, and why?

- useDeferredValue, because it wraps a value and defers the consumer rendering from it
- useTransition, because startTransition can wrap any render that is expensive, not just a setter
- useSyncExternalStore, because the prop originates outside this component's own state
- useImperativeHandle, so the parent can push the expensive update down only when it is ready

Check answer

Core hooks React 19 hooks

Frontend System Desi...

LEARN MORE

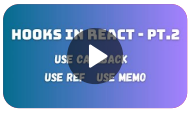
ARTICLES & DOCS

Built-in React hooks (reference)
react.dev

VIDEOS



Advanced hooks: useRef, useMemo, useCallback, useReducer
YouTube



useCallback, useMemo & useRef
YouTube



When to use useCallback & useMemo
YouTube

Hand-picked references for this lesson.